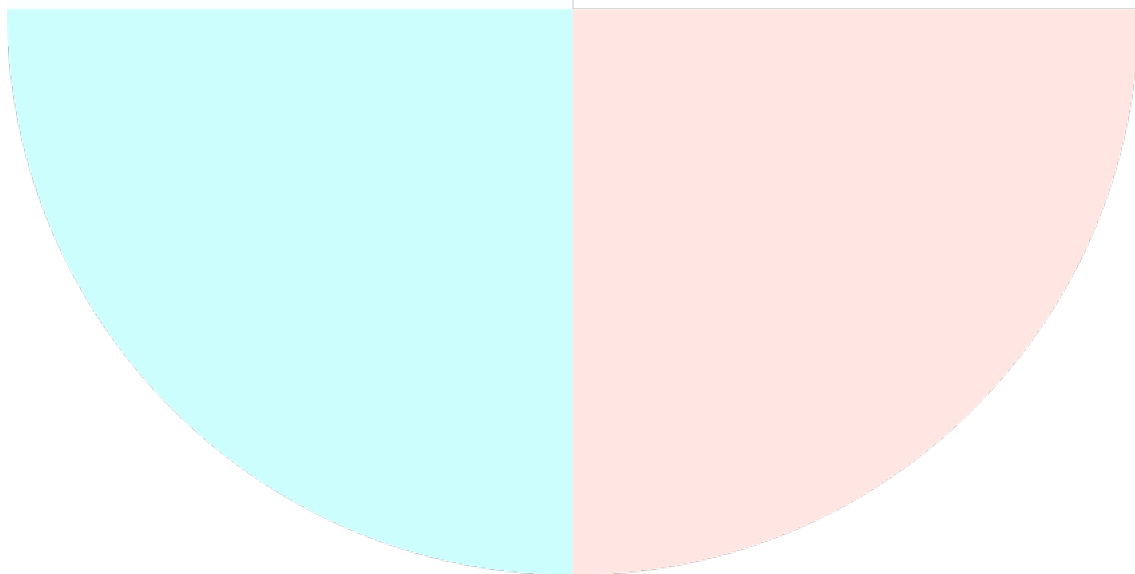


Bloque 3 - Tema 4

test

LENGUAJES DE INTERROGACIÓN DE BASES DE
DATOS. ESTÁNDAR ANSI SQL. PROCEDIMIENTOS
ALMACENADOS. EVENTOS Y DISPARADORES



PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

1. En un Sistema Gestor de Base de Datos (SGBD), el lenguaje DDL proporciona una serie de facilidades, las cuales son:
 - a) Suministra facilidades para poder consultar y actualizar los datos existentes en la Base de Datos.
 - b) Brinda facilidades para poder gestionar permisos sobre los objetos de la base de datos.
 - c) Proporciona facilidades para poder crear tablas o columnas.
 - d) Aporta facilidades para hacer SELECT sobre las tablas de la base de datos.
2. El lenguaje SQL posee una serie de operadores de agregación, cuál de los siguientes NO es uno de ellos:
 - a) LIKE.
 - b) AVG.
 - c) MIN.
 - d) COUNT.
3. En SQL, las cláusulas para, respectivamente, consultar datos, introducirlos, modificarlos y borrarlos, son:
 - a) SELECT, APPEND, UPDATE, DELETE.
 - b) SELECT, INSERT, UPDATE, ERASE.
 - c) SELECT, APPEND, UPDATE, ERASE.
 - d) SELECT, INSERT, UPDATE, DELETE.
4. La sentencia TRUNCATE del lenguaje SQL pertenece al:
 - a) DDL.
 - b) DCL.
 - c) DCU.
 - d) DML.
5. Sean dos tablas relacionadas, A y B, en un SGBD relacional. En A la clave primaria es el campo llamado "primaria" y, en B, la clave foránea (referenciada con A) es el campo "foránea". Si queremos obtener mediante un JOIN las filas de A junto con las filas de B correspondientes, y además también las filas de A que no tengan correspondencias en B, ¿qué usaríamos?
 - a) A LEFT JOIN B ON A.primaria = B.foranea.
 - b) A INNER JOIN B ON A.primaria = B.foranea.
 - c) A RIGHT JOIN B ON A.primaria = B.foranea.
 - d) A OUTER JOIN B ON A.primaria = B.foranea.
6. En SQL, para especifica una condición que debe cumplir un grupo de agregación, usamos:
 - a) GROUP BY.
 - b) COUNT.
 - c) SELECT.
 - d) HAVING.

7. La sentencia SQL que nos permite generar una vista de una tabla es:
- DEFINE VIEW.
 - DROP VIEW.
 - CREATE VIEW.
 - NEW VIEW.
8. Señale la opción INCORRECTA con respecto de las restricciones de integridad (CONSTRAINTS) en SQL:
- Pueden definirse en la creación de la tabla.
 - Pueden definirse después de que la tabla haya sido creada.
 - Se definen mediante la sentencia CREATE RESTRICTION.
 - FOREIGN KEY y NOT NULL son dos tipos de restricciones.
9. El estándar ANSI SQL especifica estos tipos de cláusulas JOIN:
- JOIN, LEFT/RIGHT [INNER], COMPLETE y CROSS.
 - INNER, LEFT/RIGHT/FULL [OUTER] y CROSS.
 - INNER, LEFT/RIGHT/BOTH [OUTER] y ACROSS.
 - INSIDE, OUTSIDE y LEFT/RIGHT/FULL [OUTER].
10. En una base de datos relacional se quiere añadir una nueva columna en una tabla ya existente. ¿Qué sentencia SQL habría que utilizar?
- Add column.
 - Alter table.
 - Insert column.
 - Modify column.
11. En cuanto al lenguaje de interrogación de bases de datos SQL:
- Es un estándar de facto cuando hablamos de lenguajes de interrogación de bases de datos y su base es el álgebra relacional.
 - Es un lenguaje de interrogación de bases de datos utilizado para hacer consultas sobre bases de datos estructuradas, relacionales, jerárquicas o en red.
 - Es un lenguaje de consulta, pero para realizar otras operaciones sobre bases de datos como la definición/creación de las mismas se utiliza otro tipo de lenguajes.
 - No está implementado en gestores de bases de datos menores como Microsoft Access.
12. ¿Cuál de los siguientes términos se relaciona directamente con el control de integridad en SQL?
- ROLLBACK.
 - GRANT.
 - CREATE INDEX.
 - REVOKE.
13. ¿Cuál es el propósito del lenguaje DDL en una base de datos?
- Acceso a la base de datos de lectura pero no de escritura.
 - Acceder a la base de datos para leer, escribir o modificar la información.
 - Recuperar y eliminar una base de datos.
 - Definir la estructura lógica de una base de datos.

14. ¿Qué sentencia de las siguientes pertenece a la categoría de DDL?

- a) CREATE.
- b) UPDATE.
- c) SELECT.
- d) DELETE.

15. ¿Qué operador de SQL, utilizado junto a la cláusula WHERE, permite buscar según un patrón determinado en una columna?

- a) LIKE.
- b) IN.
- c) BETWEEN.
- d) HAVING.

16. Indicar qué resultado se puede esperar de esta consulta SQL sobre una tabla COCHES_VENTA que mantiene el inventario de automóviles en un pequeño concesionario:

```
SELECT Modelo, Color, COUNT(Bastidor) AS Num  
FROM COCHES_VENTA  
GROUP BY Modelo, Color  
HAVING COUNT(Bastidor) <= 1
```

- a) Le indica al dueño del concesionario que quizá debe plantearse adquirir más existencias de un cierto modelo y color de automóvil.
- b) Le indica al dueño del concesionario todos los modelos distintos del concesionario. Es decir, un inventario organizado por modelos.
- c) Le indica al dueño del concesionario todos los modelos distintos del concesionario. Es decir, un inventario organizado por modelos y color.
- d) Le indica al dueño todos los modelos cuyo número de bastidor es menor o igual a 1.

17. Indique la cláusula necesaria para crear el esquema de la base de datos museoDB y hacer que todos los objetos creados en el mismo tengan como propietario al usuario museo:

- a) CREATE SCHEMA museoDB ON ROLE museo.
- b) CREATE SCHEMA museoDB ON USER museo.
- c) CREATE SCHEMA museoDB TO USER museo.
- d) CREATE SCHEMA museoDB AUTHORIZATION museo.

18. Después de crear el usuario de la BD museo_consulta, se necesita que se le asigne permisos de SELECT en la tabla ENTIDADES_EXTERNAS. La sentencia para ello sería:

- a) GRANT SELECT IN TABLE ENTIDADES_EXTERNAS TO museo_consulta.
- b) GRANT SELECT ON ENTIDADES_EXTERNAS TO museo_consulta.
- c) GRANT USAGE SELECT TO ENTIDADES_EXTERNAS ON museo_consulta.
- d) GRANT SELECT TO ENTIDADES_EXTERNAS ON museo_consultas.

19. Insertar una constraint en la cláusula SQL de creación de la tabla RESERVAS para el campo de clave ajena idcliente, de forma que NO ejecute ninguna acción en la tabla referenciada en los casos de UPDATE o DELETE:

- a) CONSTRAINT con_fk FOREIGN KEY (idcliente) ON CLIENTES (id) MATCH SIMPLE WITH NO ACTION.
- b) CONSTRAINT con_fk FOREIGN KEY ON idcliente MATCH SIMPLE WITH CLIENTES (id) ON UPDATE NO ACTION ON DELETE NO ACTION.
- c) CONSTRAINT con_fk FOREIGN KEY (idcliente) REFERENCES CLIENTES (id) ON UPDATE NO ACTION ON DELETE NO ACTION.
- d) CONSTRAINT con_fk FOREIGN KEY (idcliente) ON CLIENTES (id) MATCH SIMPLE WITH UPDATE NO ACTION WITH DELETE NO ACTION.

20. En SQL, ¿cuál de las siguientes opciones representa la secuencia de parámetros en el orden correcto deberían aparecer en una sentencia?

- a) SELECT – FROM – WHERE – ORDER BY – GROUP BY – HAVING.
- b) SELECT – FROM – WHERE – GROUP BY – HAVING – ORDER BY.
- c) SELECT – FROM – WHERE – GROUP BY – ORDER BY – HAVING.
- d) SELECT – FROM – WHERE – ORDER BY – HAVING – GROUP BY.

21. La sintaxis estándar de ANSI SQL que nos permite borrar una vista es:

- a) ERASE VIEW.
- b) DELETE VIEW.
- c) CLEAR VIEW.
- d) DROP VIEW.

22. Indique qué sentencia ANSI SQL utilizaría para deshacer una transacción:

- a) ROLLBACK.
- b) BACKSTEP.
- c) REVOKE.
- d) UNDO.

23. Seleccione la respuesta correcta en relación a las bases de datos relacionales:

- a) El lenguaje que se usa para manipular datos (seleccionar, borrar, etc.) es diferente que el que se usa para definir datos (crear/modificar tablas, etc.).
- b) En el lenguaje SQL, el operador BETWEEN no puede estar en una sentencia WHERE.
- c) En el lenguaje SQL, la sintaxis para borrar una tabla es: DELETE TABLE (NOMBRE_TABLE).
- d) Las claves ajenas (foreign keys) pueden ser nulas.

24. ¿Cuál de las siguientes sentencias forma parte del DDL de SQL?

- a) INSERT.
- b) UPDATE.
- c) ALTER.
- d) COMMIT.

25. En SQL, la cláusula having:

- a) Se utiliza específicamente para realizar cálculos con campos tipo Datetime.
- b) Se usa habitualmente en combinación con la cláusula group by.
- c) El uso de having impide usar la cláusula where en la misma sentencia.
- d) Es un comando que se incluye dentro del llamado DDL.

26. Con la cláusula ORDER BY de SQL, si nos encontramos con la siguiente consulta: SELECT * FROM Empleados ORDER BY Provincia DESC, Municipio. ¿Cuál es el resultado que se obtendría?

- a) Un listado de empleados ordenado de manera ascendente por la columna Provincia y, dentro de cada provincia ordenado de manera ascendente por la columna Municipio.
- b) Un listado de empleados ordenado de manera descendente por la columna Provincia y, dentro de cada provincia ordenado de manera ascendente por la columna Municipio.
- c) Un listado de empleados ordenado de manera ascendente por la columna Provincia y, dentro de cada provincia ordenado de manera descendente por la columna Municipio.
- d) Da error.

27. ¿Cuál de las siguientes sentencias utilizaríamos para permitir al USUARIO1 actualizar la columna SALARIO de la tabla EMPLEADOS sin permitirle dar de alta nuevos empleados?

- a) GRANT EMPLEADOS ON SELECT, UPDATE(SALARIO) TO USUARIO1.
- b) GRANT ALL ON SELECT, UPDATE TO USUARIO1.
- c) GRANT ALL ON EMPLEADOS TO USUARIO1.
- d) GRANT SELECT, UPDATE(SALARIO) ON EMPLEADOS TO USUARIO1.

28. De acuerdo al estándar ANSI SQL, ¿cuál de las siguientes opciones es equivalente a la operación JOIN?

- a) LEFT JOIN.
- b) FULL JOIN.
- c) INNER JOIN.
- d) OUTER JOIN.

29. ¿Cuál de las siguientes sentencias SQL es correcta?

- a) DROP TABLE t WHERE a=1;
- b) TRUNCATE TABLE t WHERE a=1;
- c) DELETE FROM t;
- d) DELETE FROM t1, t2 WHERE t1.a=t2.a;

Dado el siguiente esquema de bases de datos:

- Proveedores, S=(S#, nomprov, domicilio, ciudad)
- Piezas, P=(P#, nompiez, color)
- Proyectos, J=(J#, nomproy, duración, ciudad)
- Envíos o Suministros, que relaciona a los proveedores que suministran ciertas cantidades de piezas a los proyectos, SPJ=(S#, P#, J#, cantidad)

Responda a las siguientes cuestiones:

30. ¿Cuál de las siguientes consultas en SQL contesta a la pregunta "Eliminar todos los proveedores que no realicen envíos"?

- a) DELETE
FROM S
WHERE S# NOT IN (SELECT S# FROM SPJ);
- b) DELETE S#
FROM S
WHERE S IN (SELECT S# FROM SPJ);
- c) DELETE S#
FROM S
WHERE S# NOT IN (SELECT S# FROM SPJ);
- d) Ninguna de las respuestas anteriores es correcta.

31. ¿Qué realiza la siguiente sentencia?

- ```
DELETE
FROM J
WHERE J# NOT IN (SELECT J# FROM SPJ);
```
- a) Eliminar todos los proyectos para los cuales no existan envíos.
  - b) Eliminar todos los proyectos que tienen algún envío.
  - c) Eliminar todos los proyectos.
  - d) Ninguna de las respuestas anteriores es correcta.

**32. ¿Qué permite obtener la siguiente consulta?**

- ```
SELECT DISTINCT J#  
FROM J  
WHERE ciudad="Madrid"  
AND S# IN (SELECT J# FROM J WHERE ciudad = "Barcelona");
```
- a) Obtener los J# de los proyectos que son de "Madrid" o de "Barcelona".
 - b) Obtener los J# de los proyectos que son de "Madrid" y algunos de "Barcelona".
 - c) Obtener los J# de los proyectos que son de "Madrid" y de "Barcelona".
 - d) Ninguna de las respuestas anteriores es correcta.

33. ¿Qué permite obtener la siguiente consulta?

- ```
SELECT DISTINCT J#
FROM J
WHERE ciudad="Madrid"
AND J# IN (SELECT J# FROM J WHERE ciudad = "Barcelona");
```
- a) Obtener los J# de los proyectos que son de "Madrid" o de "Barcelona".
  - b) Obtener los J# de los proyectos que son de "Madrid" y algunos de "Barcelona".
  - c) Obtener los J# de los proyectos que son de "Madrid" y de "Barcelona".
  - d) Ninguna de las respuestas anteriores es correcta.

34. ¿Cuál de las siguientes consultas SQL contesta a la pregunta “Obtener los S# de los proveedores que suministren por lo menos alguna de las piezas suministradas por alguno de los proveedores que suministran alguna pieza”?

- a) 

```
SELECT DISTINCT S#
FROM SPJ
WHERE P# IN (SELECT a.P# FROM SPJ a
 WHERE a.S# IN (SELECT b.S# FROM SPJ b
 WHERE b.P# IN (SELECT P.P# FROM P))));
```
- b) 

```
SELECT DISTINCT S#
FROM SPJ
WHERE P# IN (SELECT P# FROM P
 WHERE S# IN (SELECT S# FROM S
 WHERE P# IN (SELECT P# FROM P))));
```
- c) 

```
SELECT DISTINCT S#
FROM SPJ
WHERE P# IN (SELECT P# FROM P);
```
- d) Ninguna de las respuestas anteriores es correcta.

35. ¿Cuál de las siguientes consultas SQL contesta a la pregunta “Obtener los J# de los proyectos donde se utilice al menos una de las piezas suministrada por los proveedores S#='S1' o S#='S2'”?

- a) 

```
SELECT DISTINCT SPJ.J#
FROM SPJ
WHERE EXISTS (SELECT * FROM SPJ
 WHERE SPJ.P#=P# AND S#='S1' OR S#='S2');
```
- b) 

```
SELECT DISTINCT SPJX.J#
FROM SPJ
WHERE S#='S1' OR s#='S2';
```
- c) 

```
SELECT DISTINCT J#
FROM SPJ
WHERE EXISTS (SELECT * FROM SPJ
 WHERE PJ.P#='S1' OR s#='S2');
```
- d) 

```
SELECT DISTINCT SPJX.J#
FROM SPJ SPJX
WHERE EXISTS (SELECT * FROM SPJ
 WHERE SPJ.P#=SPJX.P# AND SPJ.S#='S1' OR SPJ.S#='S2');
```



36. ¿Cuál de las siguientes consultas en SQL contesta a la pregunta "Obtener los J# de los proyectos para los cuales el proveedor S#='S1' es el único proveedor?"

- a) SELECT DISTINCT J#  
FROM SPJ SPJX  
WHERE NOT EXISTS (SELECT \* FROM SPJ  
WHERE SPJ.J#=SPJX.J# AND SPJ.S# <> 'S1');
- b) SELECT DISTINCT J#  
FROM SPJ SPJX  
WHERE EXISTS (SELECT \* FROM SPJ  
WHERE SPJ.J#=SPJX.J# AND SPJ.S# = 'S1');
- c) SELECT DISTINCT J#  
FROM SPJ SPJX  
WHERE EXISTS (SELECT \* FROM SPJ  
WHERE SPJ.S# = 'S1');
- d) SELECT DISTINCT J#  
FROM SPJ SPJX  
WHERE EXISTS (SELECT \* FROM SPJ  
WHERE SPJ.P#=SPJX.P# AND SPJ.S# <> 'S1');

37. ¿Cuál de las respuestas es contestada por la siguiente consulta SQL?

```
SELECT DISTINCT P#
FROM SPJ SPJX
WHERE NOT EXISTS (SELECT * FROM J
WHERE CIUDAD='Londres' AND
NOT EXISTS (SELECT * FROM SPJ SPJY
WHERE SPJY.P# = SPJX.P# AND
SPJY.J# = J.J#));
```

- a) Obtener los P# de las piezas suministradas a todos los proyectos que no son de 'Londres'.
- b) Obtener los P# de las piezas suministradas solamente a los proyectos de 'Londres'.
- c) Obtener los P# de las piezas suministradas a todos los proyectos de 'Londres'.
- d) Ninguna respuesta responde a la cuestión planteada.

**38. ¿Cual de las siguientes consultas en SQL contesta a la pregunta "Obtener los P# de las piezas suministradas a algún proyecto tales que la cantidad media suministrada en total sea mayor que 320"?**

- a) SELECT DISTINCT P#  
FROM SPJ  
GROUP BY P#  
WHERE AVG(cantidad) > 320;
- b) SELECT DISTINCT P#  
FROM SPJ  
WHERE AVG(cantidad) > 320;  
GROUP BY P#
- c) SELECT DISTINCT P#  
FROM SPJ  
GROUP BY P#, J#  
HAVING AVG(cantidad) > 320;
- d) SELECT DISTINCT P#  
FROM SPJ  
GROUP BY J#  
HAVING AVG(cantidad) > 320;

**39. En un SGBD Oracle, indique cuál de los siguientes es un lenguaje de procedimiento cuya sintaxis permite insertar sentencias SQL y se almacena compilado dentro de la base de datos:**

- a) TRANSACT SQL.
- b) PL/SQL.
- c) FORTRAN.
- d) COBOL.

**40. En SQL, ¿cómo se pueden eliminar los datos en una tabla, pero no la propia definición de la tabla?**

- a) DROP TABLE.
- b) DELETE.
- c) REMOVE.
- d) ERASE.

**41. Un analista debe definir una columna para almacenar cadenas de texto de longitud variable, hasta un máximo de 100 caracteres, en una tabla SQL estándar. ¿Cuál es el tipo de dato más adecuado según el estándar ANSI SQL?**

- a) CHAR(100).
- b) VARCHAR(100).
- c) TEXT.
- d) STRING(100).

- 42. En SQL estándar, ¿qué ocurre si se ejecuta DROP SCHEMA mi\_esquema RESTRICT; y el esquema contiene tablas?**
- a) El esquema se elimina solo si no contiene elementos.
  - b) El esquema y todas sus tablas se eliminan.
  - c) El esquema se elimina y las tablas se renombran.
  - d) El comando falla y elimina solo las tablas.
- 43. En la definición de una clave foránea en SQL, ¿qué opción permite que, al eliminar una fila referenciada, las filas hijas se eliminen automáticamente?**
- a) ON DELETE NO ACTION.
  - b) ON DELETE SET NULL.
  - c) ON DELETE CASCADE.
  - d) ON DELETE SET DEFAULT.
- 44. ¿Cuál de las siguientes afirmaciones sobre las vistas en SQL es correcta?**
- a) Una vista siempre almacena físicamente los datos que muestra.
  - b) Una vista no puede tener restricciones de inserción o modificación.
  - c) Una vista no puede usarse para simplificar consultas complejas.
  - d) Una vista puede ser actualizable solo si se basa en una única tabla sin GROUP BY ni HAVING.
- 45. ¿Cuál es la forma correcta de insertar una fila en la tabla EMPLEADOS, especificando solo los campos NOMBRE y SUELDO, dejando el resto con valores por defecto?**
- a) INSERT INTO EMPLEADOS VALUES ('Ana', 2000);
  - b) INSERT INTO EMPLEADOS (NOMBRE, SUELDO) VALUES ('Ana', 2000);
  - c) INSERT EMPLEADOS (NOMBRE, SUELDO) VALUES ('Ana', 2000);
  - d) INSERT INTO EMPLEADOS SET NOMBRE='Ana', SUELDO=2000;
- 46. ¿Cuál de las siguientes consultas obtiene los nombres de los proyectos donde todos los empleados asignados tienen un sueldo inferior al precio del proyecto?**
- a) SELECT nombre\_proyecto FROM proyectos WHERE precio < ANY (SELECT sueldo FROM empleado WHERE proyectos.idproyecto = empleado.idproyecto);
  - b) SELECT nombre\_proyecto FROM proyectos WHERE precio > ALL (SELECT sueldo FROM empleado WHERE proyectos.idproyecto = empleado.idproyecto);
  - c) SELECT nombre\_proyecto FROM proyectos WHERE precio = ALL (SELECT sueldo FROM empleado WHERE proyectos.idproyecto = empleado.idproyecto);
  - d) SELECT nombre\_proyecto FROM proyectos WHERE precio < ALL (SELECT sueldo FROM empleado WHERE proyectos.idproyecto = empleado.idproyecto);
- 47. ¿Qué tipo de JOIN permite obtener todas las filas de la tabla izquierda y solo las coincidentes de la derecha, rellenando con NULL donde no hay coincidencia?**
- a) INNER JOIN.
  - b) RIGHT JOIN.
  - c) LEFT JOIN.
  - d) FULL JOIN.

**48. ¿Qué sentencia permite otorgar a un usuario el privilegio de seleccionar datos de una tabla y, además, permitirle conceder ese privilegio a otros usuarios?**

- a) GRANT SELECT ON tabla TO usuario;
- b) GRANT ALL ON tabla TO usuario;
- c) GRANT USAGE ON tabla TO usuario;
- d) GRANT SELECT ON tabla TO usuario WITH GRANT OPTION;

**49. En SQL Server, ¿cómo se define un parámetro de salida en un procedimiento almacenado?**

- a) @parametro INT OUT
- b) parametro OUT INT
- c) OUT @parametro INT
- d) @parametro OUTPUT INT

**50. ¿Cuál de las siguientes afirmaciones sobre triggers es correcta?**

- a) Un trigger solo puede ejecutarse antes de una operación (BEFORE).
- b) Un trigger puede ejecutarse antes (BEFORE), después (AFTER) o en sustitución (INSTEAD OF) de una operación.
- c) Un trigger solo puede asociarse a operaciones de inserción.
- d) Un trigger no puede acceder a los valores antiguos y nuevos de una fila.

**51. ¿Cuál es la principal ventaja de utilizar un driver JDBC tipo 4 (Java puro) frente a los demás tipos?**

- a) Permite acceder a cualquier SGBD sin instalar drivers específicos.
- b) Es el más eficiente, ya que no requiere capas intermedias y traduce directamente las llamadas JDBC al protocolo del SGBD.
- c) Requiere siempre un middleware adicional.
- d) Solo funciona con bases de datos no relacionales.

Disponemos de dos tablas:

- empleados(id, empleado, nombre, id\_departamento)
- departamentos(id\_departamento, nombre\_departamento)

**52. Se quiere obtener el nombre de cada empleado junto al nombre de su departamento, solo para aquellos empleados que tienen departamento asignado. ¿Qué consulta devuelve la información requerida?**

- a) SELECT nombre, nombre\_departamento FROM empleados, departamentos;
- b) SELECT nombre, nombre\_departamento FROM empleados INNER JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- c) SELECT nombre, nombre\_departamento FROM empleados LEFT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- d) SELECT nombre, nombre\_departamento FROM empleados RIGHT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;

**53. Se necesitan obtener todos los empleados, incluyendo aquellos que no tienen departamento asignado, mostrando NULL en el nombre del departamento si no existe. ¿Qué consulta devuelve la información que se necesita?**

- a) SELECT nombre, nombre\_departamento FROM empleados LEFT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- b) SELECT nombre, nombre\_departamento FROM empleados RIGHT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- c) SELECT nombre, nombre\_departamento FROM empleados INNER JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- d) SELECT nombre, nombre\_departamento FROM empleados, departamentos;

**54. Es necesario obtener todos los departamentos, incluyendo aquellos que no tienen empleados asignados, mostrando NULL en el nombre del empleado si no existe. ¿Qué consulta**

- a) SELECT nombre, nombre\_departamento FROM empleados RIGHT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- b) SELECT nombre, nombre\_departamento FROM empleados LEFT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- c) SELECT nombre, nombre\_departamento FROM empleados INNER JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- d) SELECT nombre, nombre\_departamento;

**55. Se precisa obtener todos los empleados y todos los departamentos, mostrando NULL donde no haya coincidencia entre empleado y departamento.**

- a) SELECT nombre, nombre\_departamento FROM empleados FULL OUTER JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- b) SELECT nombre, nombre\_departamento FROM empleados LEFT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- c) SELECT nombre, nombre\_departamento FROM empleados RIGHT JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;
- d) SELECT nombre, nombre\_departamento FROM empleados INNER JOIN departamentos ON empleados.id\_departamento = departamentos.id\_departamento;

**56. Tienes dos tablas con una columna llamada id en ambas, y quieres hacer una combinación automática por esa columna, sin especificar ON ni USING. ¿Qué tipo de JOIN se debe usar?**

- a) INNER JOIN.
- b) NATURAL JOIN.
- c) LEFT JOIN.
- d) CROSS JOIN.

**57. Dispones de las tablas funcionarios(id\_funcionario, nombre, apellidos, id\_unidad) y expedientes (id\_expediente, id\_funcionario, fecha\_inicio, estado). Queremos saber cuántos expedientes ha tramitado cada funcionario. ¿Cuál es la consulta SQL más adecuada?**

- a) SELECT f.nombre, f.apellidos, COUNT(e.id\_expediente) AS num\_expedientes FROM funcionarios f LEFT JOIN expedientes e ON f.id\_funcionario = e.id\_funcionario GROUP BY f.nombre, f.apellidos;
- b) SELECT f.nombre, f.apellidos, COUNT(\*) AS num\_expedientes FROM expedientes e INNER JOIN funcionarios f ON e.id\_funcionario = f.id\_funcionario GROUP BY f.nombre, f.apellidos;
- c) SELECT nombre, apellidos, COUNT(id\_expediente) AS num\_expedientes FROM expedientes GROUP BY nombre, apellidos;
- d) SELECT f.nombre, f.apellidos, COUNT(e.id\_expediente) AS num\_expedientes FROM expedientes e RIGHT JOIN funcionarios f ON e.id\_funcionario = f.id\_funcionario GROUP BY f.nombre, f.apellidos;

**58. Tienes las tablas unidades(id\_unidad, nombre\_unidad, sede) y funcionarios(id\_funcionario, nombre, id\_unidad). Quieren saber cuántos funcionarios hay en cada unidad administrativa, mostrando también las unidades sin funcionarios. ¿Cuál es la consulta correcta?**

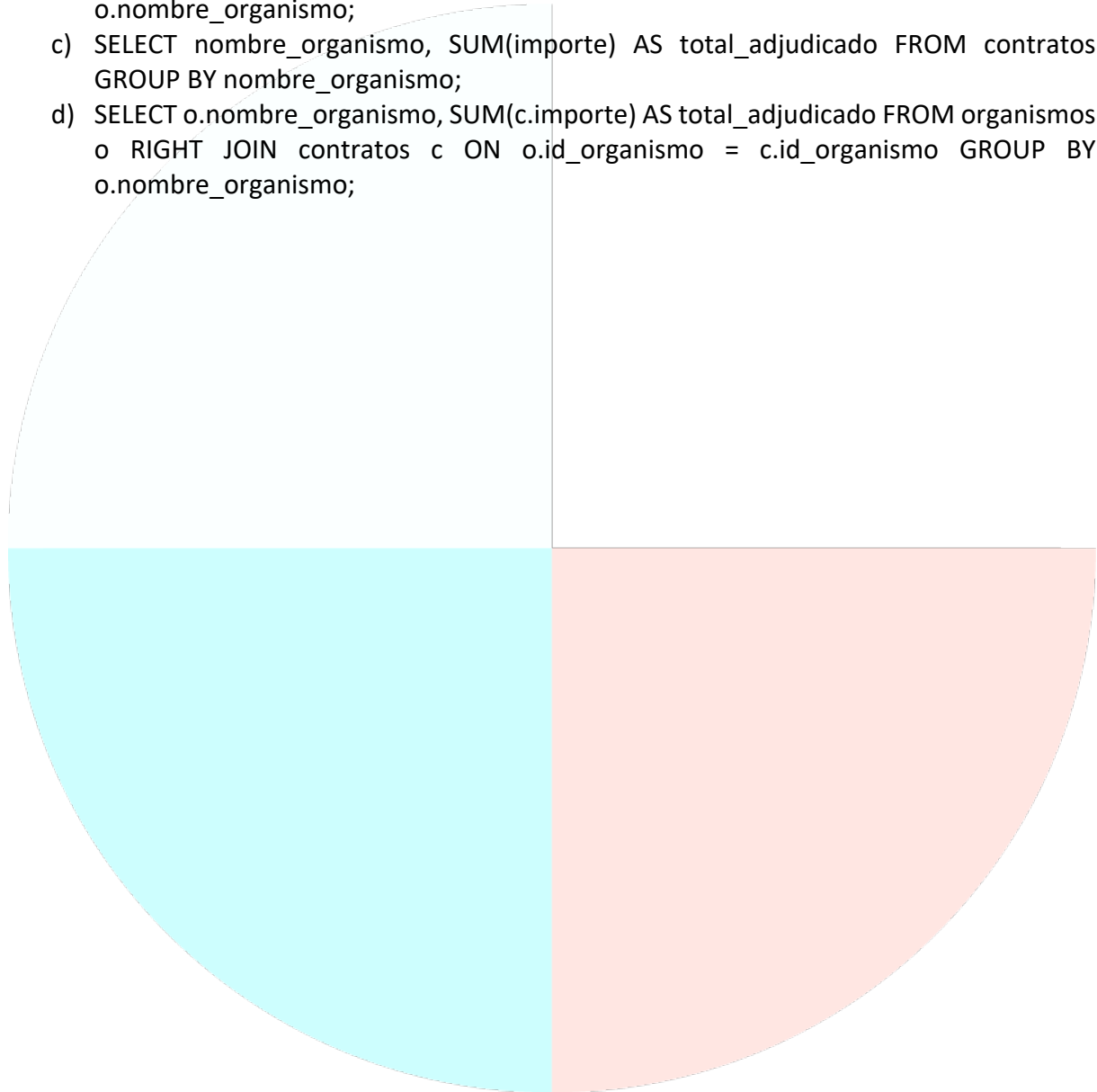
- a) A)SELECT u.nombre\_unidad, COUNT(f.id\_funcionario) AS num\_funcionarios FROM unidades u LEFT JOIN funcionarios f ON u.id\_unidad = f.id\_unidad GROUP BY u.nombre\_unidad;
- b) B) SELECT u.nombre\_unidad, COUNT(\*) AS num\_funcionarios FROM funcionarios f INNER JOIN unidades u ON f.id\_unidad = u.id\_unidad GROUP BY u.nombre\_unidad;
- c) C) SELECT u.nombre\_unidad, COUNT(f.id\_funcionario) AS num\_funcionarios FROM funcionarios f RIGHT JOIN unidades u ON f.id\_unidad = u.id\_unidad GROUP BY u.nombre\_unidad;
- d) D) SELECT nombre\_unidad, COUNT(id\_funcionario) AS num\_funcionarios FROM funcionarios GROUP BY nombre\_unidad;

**59. Tenemos de las tablas ciudadanos(dni, nombre, provincia) y solicitudes(id\_solicitud, dni, fecha, tipo\_tramite). Se requiere obtener las provincias donde los ciudadanos han presentado más de 100 solicitudes. ¿Cuál es la consulta adecuada?**

- a) SELECT c.provincia, COUNT(s.id\_solicitud) AS total\_solicitudes FROM ciudadanos c INNER JOIN solicitudes s ON c.dni = s.dni GROUP BY c.provincia HAVING COUNT(s.id\_solicitud) > 100;
- b) SELECT provincia, COUNT(id\_solicitud) AS total\_solicitudes FROM solicitudes GROUP BY provincia HAVING COUNT(id\_solicitud) > 100;
- c) SELECT c.provincia, COUNT(\*) AS total\_solicitudes FROM ciudadanos c LEFT JOIN solicitudes s ON c.dni = s.dni GROUP BY c.provincia HAVING COUNT(\*) > 100;
- d) SELECT provincia, COUNT(\*) AS total\_solicitudes FROM ciudadanos GROUP BY provincia HAVING COUNT(\*) > 100;

60. Tenemos las tablas organismos(id\_organismo, nombre\_organismo, tipo) y contratos(id\_contrato, id\_organismo, importe, fecha\_firma). Se desea saber el importe total adjudicado por cada organismo. ¿Cuál es la consulta correcta?

- a) SELECT o.nombre\_organismo, SUM(c.importe) AS total\_adjudicado FROM organismos o LEFT JOIN contratos c ON o.id\_organismo = c.id\_organismo GROUP BY o.nombre\_organismo;
- b) SELECT o.nombre\_organismo, SUM(c.importe) AS total\_adjudicado FROM contratos c INNER JOIN organismos o ON c.id\_organismo = o.id\_organismo GROUP BY o.nombre\_organismo;
- c) SELECT nombre\_organismo, SUM(importe) AS total\_adjudicado FROM contratos GROUP BY nombre\_organismo;
- d) SELECT o.nombre\_organismo, SUM(c.importe) AS total\_adjudicado FROM organismos o RIGHT JOIN contratos c ON o.id\_organismo = c.id\_organismo GROUP BY o.nombre\_organismo;



## SOLUCIONES

|       |       |
|-------|-------|
| 1. C  | 31. A |
| 2. A  | 32. D |
| 3. D  | 33. D |
| 4. A  | 34. A |
| 5. A  | 35. D |
| 6. D  | 36. A |
| 7. C  | 37. C |
| 8. C  | 38. C |
| 9. B  | 39. B |
| 10. B | 40. B |
| 11. A | 41. B |
| 12. A | 42. A |
| 13. D | 43. C |
| 14. A | 44. D |
| 15. A | 45. B |
| 16. A | 46. B |
| 17. D | 47. C |
| 18. B | 48. D |
| 19. C | 49. A |
| 20. B | 50. B |
| 21. D | 51. B |
| 22. A | 52. B |
| 23. D | 53. A |
| 24. C | 54. A |
| 25. B | 55. A |
| 26. B | 56. B |
| 27. D | 57. A |
| 28. C | 58. A |
| 29. C | 59. A |
| 30. A | 60. A |